

Destructors in C# with Examples

In this article, I am going to discuss the **Destructors in C#** with Examples. Please read our previous article where we discussed the [Static vs Non-Static Constructors in C#](#) with Examples.

What are Destructors in C#?

According to [MSDN](#), Destructors which are also called Finalizers in C# are used to perform any necessary final clean-up when a class instance is being collected by the garbage collector.

The Destructor is also a special type of method present in a class, just like a constructor, having the same name as the class name but prefixed with ~ tilde. The Constructor in C# is Explicitly called when the object of the class is created. On the other hand, the Destructor in C# is Implicitly Called when the object of the class is destroyed.

The Constructor and destructor methods will exactly have the same name as the class to which they belong. So, to differentiate between these two, a tilde (~) symbol is used just before the destructor method. The syntax is shown below.

```
class Example
{
    //Constructor
    public Example(int x, int y)
    {
        //Initialize the Members
    }

    //Destructor
    ~Example()
    {
        //Cleanup Statements
    }
}
```

Note: The most important point that you need to keep in mind is that a destructor method cannot have any parameters as well as cannot be applied with any modifiers. As the destructor is not parameterized, so we cannot overload the destructor.

When is a Destructor method Called in C#?

A destructor method gets called automatically by the garbage collector when the object of the class is destroyed. So, the point that you need to remember is that the destructor methods are automatically called by the garbage collector.

When will the Object of a Class get Destroyed in C#?

The object of a class in C# will be destroyed by the garbage collector in any of the following cases

1. At the end of the program execution, each and every object that is associated with the program will be destroyed by the garbage collector.
2. The Implicit calling of the garbage collector occurs sometime in the middle of the program execution provided the memory is full so the garbage collector will identify unused objects of the program and destroys them.
3. The Explicit calling of the garbage collector can also be done in the middle of program execution by the programmer with the help of the “**Collect()**” statement so that if there are any unused objects associated with the program will be destroyed in the middle of the program execution.

Example to Understand Destructors in C#:

In the below example, we have created one class called DestructorDemo with one constructor and one destructor. Then inside the Main method, we have created two instances of the DestructorDemo class and then made the obj1 value null which is eligible for garbage collection.

```
using System;
namespace DestructorExample
{
    class DestructorDemo
    {
        public DestructorDemo()
        {
            Console.WriteLine("Constructor Object Created");
        }
        ~DestructorDemo()
        {
            string type = GetType().Name;
            Console.WriteLine($"Object {type} is Destroyed");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            DestructorDemo obj1 = new DestructorDemo();
            DestructorDemo obj2 = new DestructorDemo();
            //Making obj1 for Garbage Collection
            obj1 = null;
            Console.ReadKey();
        }
    }
}
```

Now, when you run the above code, you will get the following output.

```
Constructor Object Created
Constructor Object Created
```

You can see the statements written inside the destructor are not printed in the output. Then you might be thinking that our destructor is not executed even though we made the obj2 value null. This is not right. The point that you need to remember is that the destructor method is implicitly called by the garbage collector and we cannot predict when it calls the destructor method. And hence you cannot see those print statements in the output. Once, the object is unused i.e. it ready for garbage collection, then it is the responsibility of the garbage collector to destroy that unused object and free the memory from the heap.

If you want, then you can also make an explicit call to the garbage collector in the middle of the application execution to destroy the unused object. To do so, we need to call the **GC.Collect** method as shown in the below example.

```
using System;
namespace DestructorExample
{
    class DestructorDemo
    {
        public DestructorDemo()
        {
            Console.WriteLine("Constructor Object Created");
        }
        ~DestructorDemo()
        {
            string type = GetType().Name;
            Console.WriteLine($"Object {type} is Destroyed");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            DestructorDemo obj1 = new DestructorDemo();
            DestructorDemo obj2 = new DestructorDemo();
            //Making obj1 for Garbage Collection
            obj1 = null;
            GC.Collect();
            Console.ReadKey();
        }
    }
}
```

Output:

```
Constructor Object Created
Constructor Object Created
Object DestructorDemo is Destroyed
```

Now, you can see that the Destructor is executed once. This is because we have made the obj1 value null. If we make both the object values null, then for each object the destructor method executes once. For a better understanding, please have a look at the below example.

```
using System;
namespace DestructorExample
{
    class DestructorDemo
    {
        public DestructorDemo()
        {
            Console.WriteLine("Constructor Object Created");
        }
        ~DestructorDemo()
        {
            string type = GetType().Name;
            Console.WriteLine($"Object {type} is Destroyed");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            DestructorDemo obj1 = new DestructorDemo();
            DestructorDemo obj2 = new DestructorDemo();
            //Making obj1 and obj2 ready for Garbage Collection
            obj1 = null;
            obj2 = null;
            GC.Collect();
            Console.ReadKey();
        }
    }
}
```

Output:

```
Constructor Object Created
Constructor Object Created
Object DestructorDemo is Destroyed
Object DestructorDemo is Destroyed
```

Can you prove the destructor method is Implicitly called by Garbage Collector?

Yes. Please have a look at the below example. In the below example, inside the Main method, we have created an instance of the DestructorDemo class and then we are making the object null and calling the GC.Collect methods three times. The point that you need to observe is how many times, the destructor method or finalizer is executed and when executed.

```
using System;
```

```

namespace DestructorExample
{
    class DestructorDemo
    {
        public DestructorDemo()
        {
            Console.WriteLine("DestructorDemo Object Created");
        }
        ~DestructorDemo()
        {
            Console.WriteLine("DestructorDemo Object Destroyed");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Method Execution Started");
            DestructorDemo obj1 = new DestructorDemo();

            //Making obj1 ready for Garbage Collection
            obj1 = null;
            GC.Collect();
            Console.WriteLine("Some Statement Executed Inside Main Method");
            obj1 = null;
            GC.Collect();
            Console.WriteLine("Some More Statement Executed Inside Main Method");
            GC.Collect();
            Console.WriteLine("Main Method Execution End");
            Console.ReadKey();
        }
    }
}

```

Output:

```

Main Method Execution Started
DestructorDemo Object Created
Some Statement Executed Inside Main Method
Some More Statement Executed Inside Main Method
Main Method Execution End
DestructorDemo Object Destroyed

```

```

using System;
namespace DestructorExample
{
    class DestructorDemo
    {
        public DestructorDemo()

```

```

    {
        Console.WriteLine("DestructorDemo Object Created");
    }
    ~DestructorDemo()
    {
        Console.WriteLine("DestructorDemo Object Destroyed");
    }
}
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Main Method Execution Started");
        DestructorDemo obj1 = new DestructorDemo();

        //Making obj1 ready for Garbage Collection
        obj1 = null;
        GC.Collect();
        Console.WriteLine("Some Statement Executed Inside Main Method");
        obj1 = null;
        GC.Collect();
        Console.WriteLine("Some More Statement Executed Inside Main Method");
        GC.Collect();
        Console.WriteLine("Main Method Execution End");
        Console.ReadKey();
    }
}
}

```

Pints to Remember while working with Destructors in C#:

Point1: Destructors (or Finalizers) cannot be defined in structs. In C#, they are only used with only classes. For a better understanding, please have a look at the below image.



```

//Class
class Sample
{
    //Destructor
    ~Sample()
    {
        //Cleanup Code
    }
}

//Struct
struct Example
{
    //Destructor
    ~Example()
    {
        * Example.~Example()
    }
}

```

Only class types can contain destructors

Point2: In a class, we can only define one Destructor (or Finalizer). That means Destructors cannot be overloaded in a class. For a better understanding, please have a look at the below code.

```
class Sample
{
    //Only One Destructor
    ~Sample()
    {
        //Cleanup Code
    }

    //Destructor Cannot be overloaded
    ~Sample(int c)
    {
        ~
    }
}
```

Point3: Destructors cannot be called Explicitly. They are invoked automatically by the garbage collector. At most what we can do is, we can request the Garbage collector to execute the Destructor by calling the **GC.Collect()** method. For a better understanding, please have a look at the below code.

```

class Sample
{
    ~Sample()
    {
        Console.WriteLine("Sample Class Destructor");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Sample obj = new Sample();
        obj = null;

        //Here, we are not invoking the Destructor
        //We are making a request to the Garbage Collector to
        //Execute the Destructor and Destroy the Object
        GC.Collect();
        Console.ReadKey();
    }
}

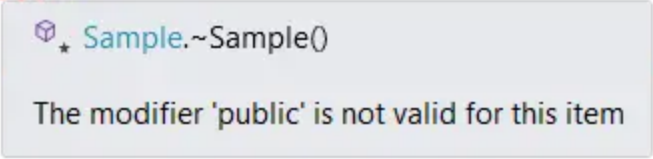
```

Point4: A Destructor does not take any modifiers or does not have any parameters. They are parameterless. For a better understanding, please have a look at the below code.

```

class Sample
{
    public ~Sample()
    {
        Console.
    }
}

```


 * Sample.~Sample()
 The modifier 'public' is not valid for this item

How Destructors are Implemented in C#?

Let us understand how Destructors are Implemented in C# with Examples. To understand how the destructor is implemented in C#, please have a look at the below example.

```

using System;
namespace DestructorExample

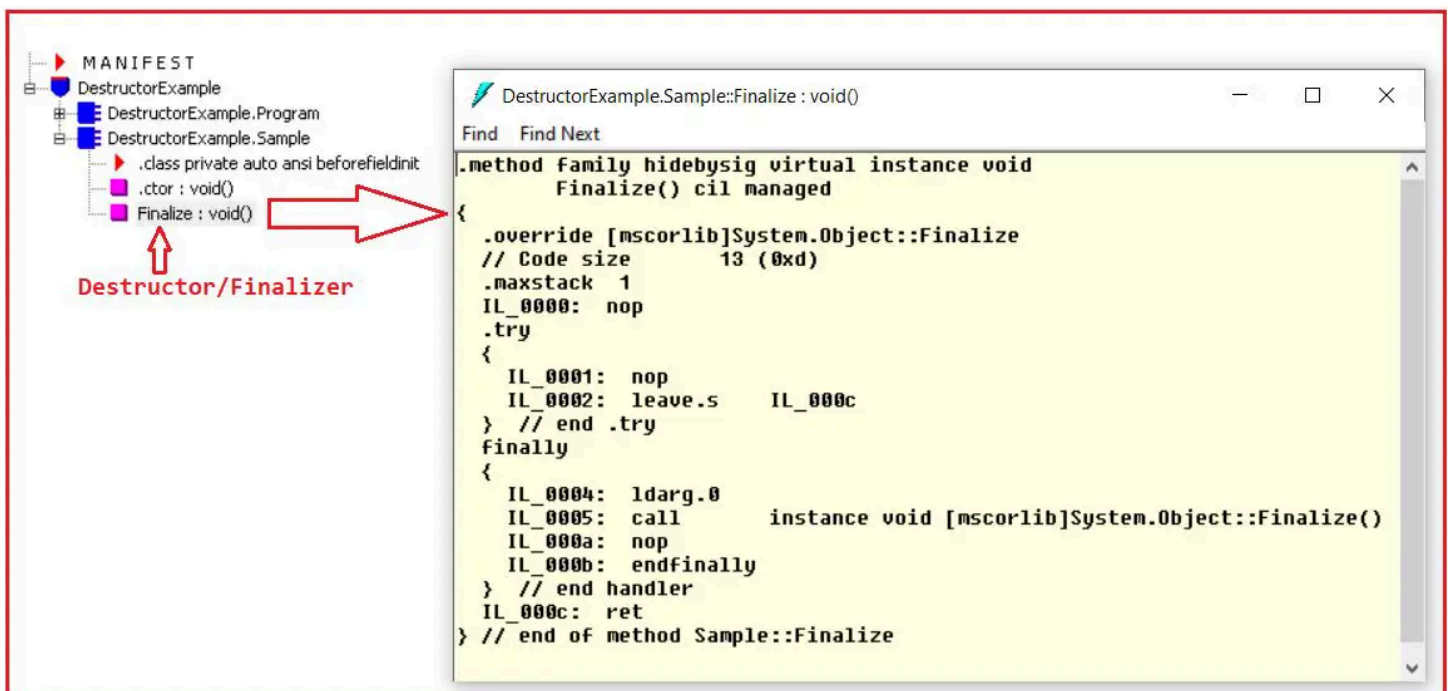
```

```

{
    class Sample
    {
        //Destructor
        ~Sample()
        {
            //Clean-up Statements
        }
    }
    class Program
    {
        static void Main()
        {
        }
    }
}

```

Now, if you check the IL Code of the above destructor using the ILDASM tool, then you will see the following IL Code is generated for the above constructor.



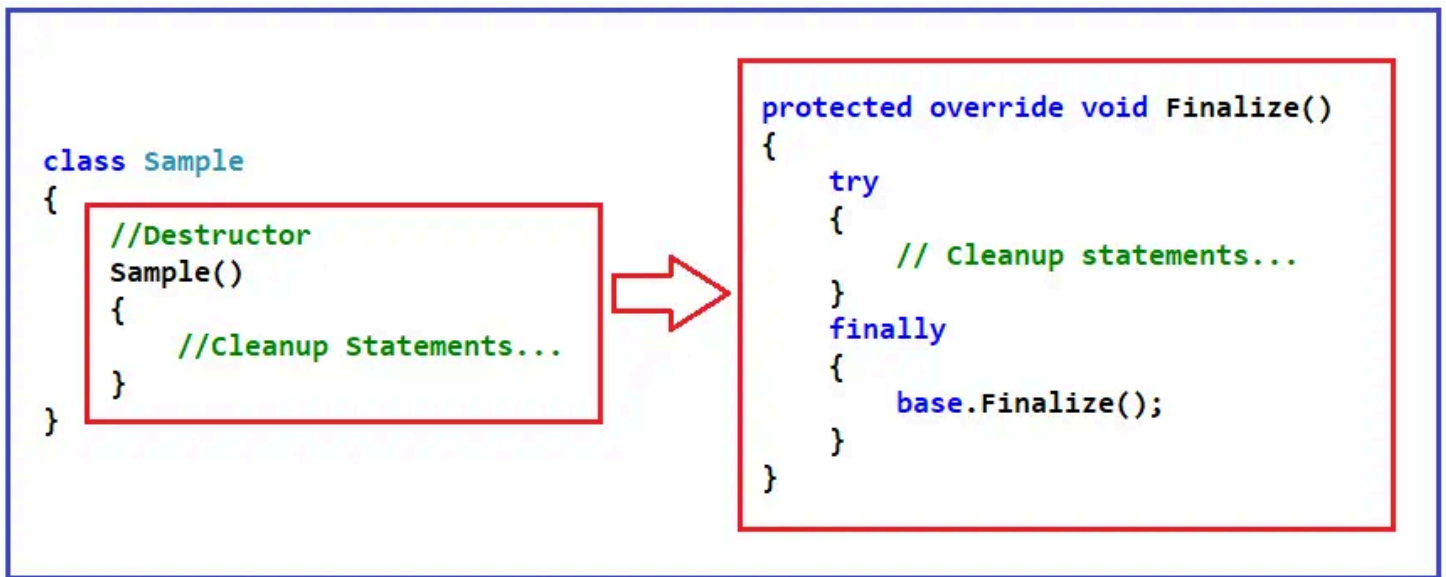
The screenshot shows the ILDASM tool interface. On the left, the project tree under 'MANIFEST' shows 'DestructorExample.Sample' selected. Below it, the methods '.ctor : void()' and 'Finalize : void()' are listed. A red arrow points from 'Finalize : void()' to the right pane. The right pane displays the IL code for 'DestructorExample.Sample::Finalize : void()'. The code is as follows:

```

.method family hidebysig virtual instance void
    Finalize() cil managed
{
    .override [mscorlib]System.Object::Finalize
    // Code size      13 (0xd)
    .maxstack 1
    IL_0000: nop
    .try
    {
        IL_0001: nop
        IL_0002: leave.s    IL_000c
    } // end .try
    finally
    {
        IL_0004: ldarg.0
        IL_0005: call      instance void [mscorlib]System.Object::Finalize()
        IL_000a: nop
        IL_000b: endfinally
    } // end handler
    IL_000c: ret
} // end of method Sample::Finalize

```

As you can see, the Destructor implicitly calls the Finalize method on the base class of the object. Therefore, a call to a destructor is implicitly translated as follows:



As you can see in the above code, the destructor is converted to Finalize method. So, whatever code you have written inside the destructor, those codes will be goes inside the try block of the Finalize method and that block code executes and clean-up the resources. Once the try block completes its execution, then the finally block is going to be executed. Finally is one block which is guaranteed to be executed. From the finally block, we are again invoking the parent class Finally method, and the parent class finalize method again going to do the same thing.

This design means that the Finalize method is called recursively for all instances in the inheritance chain, from the most-derived to the least-derived. Let us understand this with an example.

In the below example, we created three classes that make a chain of multi-level inheritance. The class First is the base class, the class Second is derived from First, and the class Third is derived from the class Second. And most importantly all these three classes have destructors. Inside the Main method, an instance of the most-derived class i.e. Third is created.

```
using System;
namespace DestructorExample
{
    public class First
    {
        ~First()
        {
            Console.WriteLine("Destructor of First Called");
        }
    }
    public class Second : First
    {
        ~Second()
        {
            Console.WriteLine("Destructor of Second Called");
        }
    }
    public class Third : Second
    {
        ~Third()
    }
}
```

```

        {
            Console.WriteLine("Destructor of Third Called");
        }
    }
}
class Program
{
    static void Main(string[] args)
    {
        Third obj = new Third();
        obj = null;
        GC.Collect();
        Console.ReadKey();
    }
}
}

```

Output:

```

Destructor of Third Called
Destructor of Second Called
Destructor of First Called

```

Note1: Empty Destructors should not be used in our application. When a class contains a destructor, then an entry for that destructor is created in the Finalize queue. This is the queue that is processed by the garbage collector. When the Garbage Collector processes the Finalize queue, it will call each destructor. So, Unnecessary destructors, including empty destructors, destructors that only call the base class destructor, or destructors that only call conditionally emitted methods, cause a needless loss of performance.

Note2: As a developer, we have no control over when the destructor is called and executed. It is the garbage collector in .NET Framework who decides when to call destructors. The garbage collector periodically checks for objects that are no longer being used by the application. If it considers an object eligible for destruction, then it calls the destructor (if any) and reclaims the memory used to store the object. It is also possible to force garbage collection by calling the **GC.Collect** method, but most of the time, this call should be avoided because it may create performance issues. [How does it impact the performance](#) that we will discuss in our next article?

When to use Destructor in C#?

You might have one question on your mind if the memory management is automatically managed by the garbage collector, then when do we need to use Destructor? In general, as C#.NET developers, we need not be much more worried about memory management. This is because the .NET garbage collector implicitly manages the allocation and deallocation of the memory for our objects.

However, when our application works with unmanaged resources, such as windows, files, and network connections, we should use a destructor to free the memory for those unmanaged resources. When the object is eligible for destruction, the garbage collector runs the Finalize method of the object. For a better understanding, please have a look at the following example.

```

using System;
namespace FileHandlinDemo
{

```

```

public class UmmanagedResource
{
    public UmmanagedResource()
    {
        //Creating Unmanaged Resources
        Console.WriteLine("Unmanaged Resource Created");
    }
    ~UmmanagedResource()
    {
        //Clean up Unmanaged Resources
        Console.WriteLine("Unmanaged Resource Destroyed");
    }
}
class Program
{
    static void Main()
    {
        UmmanagedResource resource = new UmmanagedResource();
        Console.WriteLine("Using Unmanaged Resource");
        resource = null;
        GC.Collect();

        Console.ReadKey();
    }
}

```

Output:

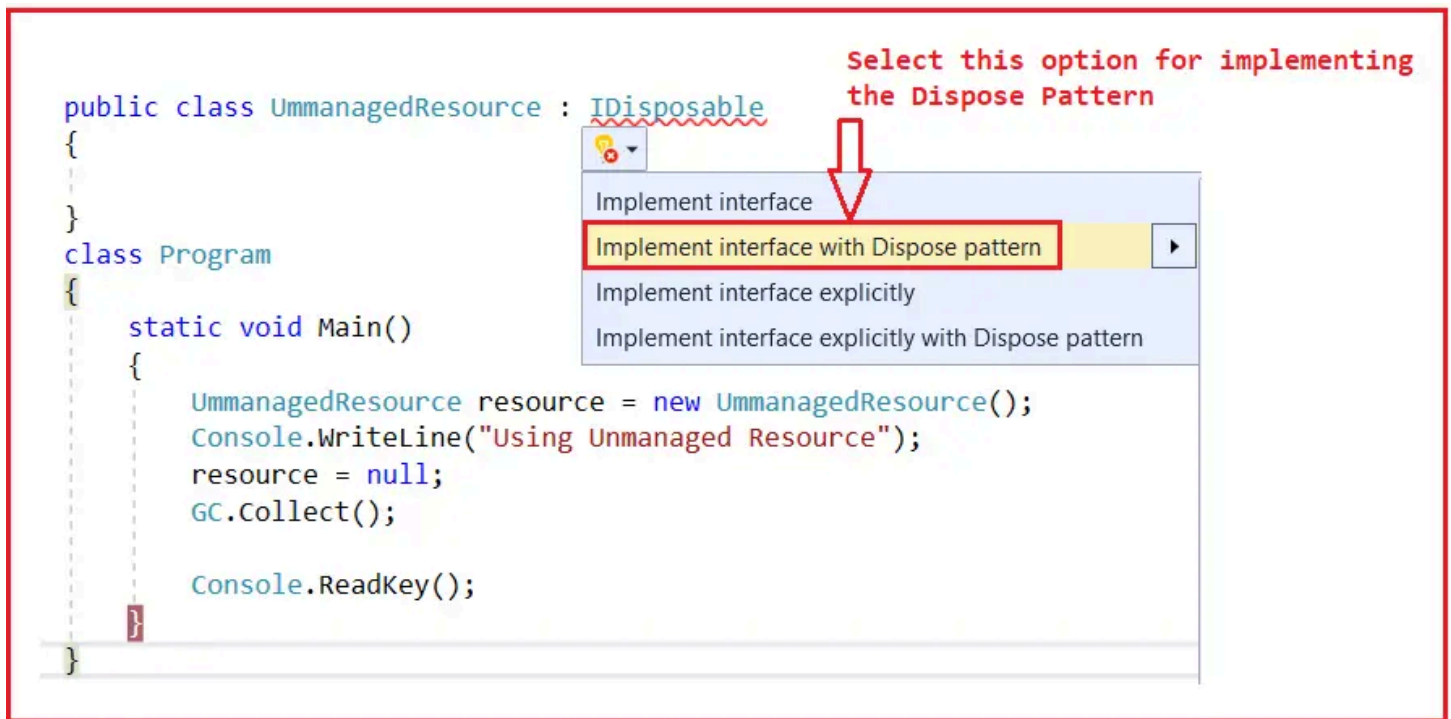
```

Unmanaged Resource Created
Using Unmanaged Resource
Unmanaged Resource Destroyed

```

Explicit Release of Resources using Dispose Pattern in C#:

If our application is using an expensive external resource, then it is recommended to provide an explicit way to release the resources before the garbage collector frees the object. To release the resource, it is recommended to implement a Dispose method of the IDisposable interface that performs the necessary clean-up for the object. So, basically, we need our class to inherit from the IDisposable interface and provide the implementation for the Dispose methods as follows. So, here, please select Implement Interface with Dispose Pattern option as shown in the below image.



Once you Implement the interface with Dispose pattern, you will see the following code, The following code is self-explained, so please go through the comment lines.

```

using System;

public class UmmanagedResource : IDisposable
{
    #region IDisposable Support
    private bool disposedValue = false; // To detect redundant calls
    protected virtual void Dispose(bool disposing)
    {
        if (!disposedValue)
        {
            if (disposing)
            {
                // TODO: dispose managed state (managed objects).
            }
            // TODO: free unmanaged resources (unmanaged objects) and override
            finalizer below.
            // TODO: set large fields to null.
            disposedValue = true;
        }
    }
    // TODO: override a finalizer only if Dispose(bool disposing) above has c
    unmanaged resources.
    // ~UmmanagedResource() {
    //     // Do not change this code. Put cleanup code in Dispose(bool disposir
    //     Dispose(false);
    // }
    // This code added to correctly implement the disposable pattern.
    public void Dispose()
    {
    }
}

```

```

        // Do not change this code. Put cleanup code in Dispose(bool disposing)
        Dispose(true);
        // TODO: uncomment the following line if the finalizer is overridden :
        // GC.SuppressFinalize(this);
    }
    #endregion
}

```

Understanding Dispose Pattern in C#:

1. **disposedValue**: The disposedValue variable to identify the redundant call. Suppose, we have requested to destroy the resources and if again we are requesting to destroy the object, then it will check this field and identify whether it is a redundant call or not, and based on this it will do the clean-up activities.
2. **Dispose(bool disposing)**: We need to write the actual logic to destroy the managed and unmanaged resources inside this method. This disposing variable value which this method takes as an input parameter will identify whether we need to destroy the managed resources or not. Further, it checks the disposedValue variable value to check whether it needs to perform the cleanup activity or not. If the request comes for the first time, the value for the disposedValue variable will be false and in that case, it will perform the clean-up activity and at the end, it sets the value true. So, when a redundant request comes, it will not perform the clean-up activity.
3. **~UmmanagedResource()**: By default, you will see that destructor or finalizer code is commented. But if you want to destroy the resources through destructor, then you need to uncomment this destructor or finalizer. But, as part of this destructor, you need to write cleanup code for managed resources and then you need to call the Dispose(false) method to clean up the unmanaged resources. You, need to override a finalizer or destructor only if Dispose(bool disposing) method has code to free unmanaged resources only.
4. **Dispose()**: This is the method that we need to call from our code to clean up the managed and unmanaged resources. As part of this method, we need to call **Dispose(true)** method which is actually going to clean up both managed and unmanaged resources. Again, if you override the destructor or finalizer method, then you need to call **GC.SuppressFinalize(this)** method which will tell the garbage collector to just ignore the destructor i.e. informing CLR (Garbage Collector) to not call the destructor or finalizer. You only need to uncomment this if you have overridden the destructor or finalizer.

The above dispose pattern can considerably improve the performance of the application. Even with this explicit control over resources, the finalizer or destructor becomes a safeguard to clean up resources if the call to the Dispose method fails. Let us write some print statements and see the output. The complete example code is given below.

```

using System;
namespace DestructorDemo
{
    public class UmmanagedResource : IDisposable
    {
        #region IDisposable Support
        private bool disposedValue = false; // To detect redundant calls
        protected virtual void Dispose(bool disposing)
        {
            if (!disposedValue)
            {
                if (disposing)
                {

```

```

        //Write Code Here to Destroy the Managed Resources
        Console.WriteLine("Managed Resources Destroyed by Dispose
    }
    //Write Code Here to Destroy the Umanaged Resources
    Console.WriteLine("Unmanaged Resources Destroyed by Dispose Me
disposedValue = true;
    }
    else
    {
        Console.WriteLine("Resources are Already Destroyed by Dispose
    }
}
~UmmanagedResource()
{
    //Write Code here to Destroy the Object
    Console.WriteLine("Request Comes to Destructor to Destroy the Resc
    Dispose(false);
}
// This code added to correctly implement the disposable pattern.
public void Dispose()
{
    //Write Code here to Destroy the Object
    Console.WriteLine("Request Comes to Dispose Method to Destroy the
Resources");
    Dispose(true);
    GC.SuppressFinalize(this);
}
#endregion
}
class Program
{
    static void Main()
    {
        UmmanagedResource resource = null;
        try
        {
            resource = new UmmanagedResource();
            Console.WriteLine("Using Resources");
        }
        finally
        {
            if (resource != null)
            {
                Console.WriteLine("Calling Dispose Method to Destroy Resou
                resource.Dispose();
            }
        }
        //Trying to Call the Dispose Method again
        Console.WriteLine();
        Console.WriteLine("Trying to Call the Dispose Method Again To Dest
Resources");
    }
}

```

```
        resource.Dispose();  
        Console.ReadKey();  
    }  
}
```

Output:

```
Using Resources  
Calling Dispose Method to Destroy Resources  
Request Comes to Dispose Method to Destroy the Resources  
Managed Resources Destroyed by Dispose Method  
Unmanaged Resources Destroyed by Dispose Method  
  
Trying to Call the Dispose Method Again To Destroy Resources  
Request Comes to Dispose Method to Destroy the Resources  
Resources are Already Destroyed by Dispose Method
```

In the next article, I am going to discuss [How Garbage Collection works in .NET Framework](#) with Examples. Here, in this article, I try to explain the **Destructor in C#** with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this Destructor in C# with Examples article.